

Rung 14 — LoRA on TRELLIS.2’s texture-flow cross-attention

Goal. One fixed mesh, a video, a temporally coherent series of textures — with the edit made *before* TRELLIS’s 3D propagation so it reaches surface the camera never saw.

1. Why not where we had it

rung13 put the adapter in the **mesh decoder**. Measured on that run:

vertices the training camera sees	16.8 %
adapter’s edit strength, unseen vs seen	1.02 — identical
R ² of the edit vs the camera’s image axis	0.16 - 0.23 (frozen colour: ~0.00)

The decoder has **no cross-attention** — the image never reaches it, and all of TRELLIS’s 3D reasoning has already finished upstream. So the adapter could only learn a function of voxel features fitted to one view. It learned a projection.

2. Where the LoRA goes

tex_slat_flow_model_1024.blocks[0 ... 29]

cross_attn.to_q	Linear(1536, 1536)	← LoRA	image → 3D k/v
cross_attn.to_kv	Linear(1024, 3072)	← LoRA	
cross_attn.to_out	Linear(1536, 1536)	← LoRA	
self_attn.to_qkv	Linear(1536, 4608)	FROZEN	3D coherence
self_attn.to_out	Linear(1536, 1536)	FROZEN	
mlp, adaLN_modulation		FROZEN	

rank 4 → 40,960 per block × 30 = ****1,228,800**** parameters. B is zero-init, so step 0 is the unmodified model.

Why cross-attention and not the MLP. A scoping decision, not a principled one — the MLP would also propagate, since 29 blocks of self-attention follow it. Cross-attention is the semantically targeted point (where image evidence enters 3D) and the one Fuse3D validated. --targets cross+mlp turns the ablation on. Note rung5_colonly *did* adapt the MLP alongside attention and reached 21.3 dB, so MLP adaptation is demonstrably not harmful — just never isolated.

Why self-attention stays frozen. It is the mechanism that makes the output 3D coherent. The whole bet is that steering *what enters* lets TRELLIS’s own propagation distribute the edit; disturbing the propagation itself would defeat that.

3. Workflow

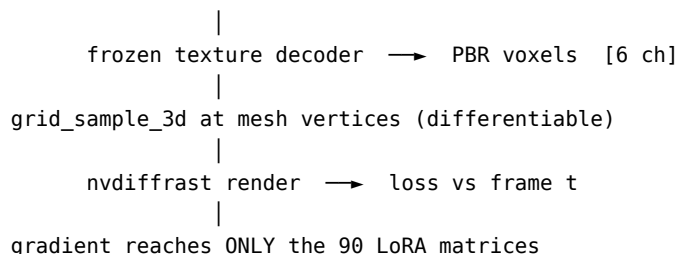
ONCE

mesh (fixed, supplied) → encode_shape_slat → shape latent [32 ch]
frames 1...150 → DIN0v3 → image tokens [1024]

PER TRAINING STEP, frame t

sample a random timestep k

run the ODE $1 \rightarrow k$	under no_grad	~6 evals avg
ONE flow evaluation at k	WITH grad	
30 blocks: self_attn frozen		
cross_attn + LoRA 6		
$\hat{x}_0 = \text{sampler}._pred_to_xstart(x_k, t_k, v)$		



Supervision: the video frame itself, one fixed camera — same signal as rung13. Only the adapter’s *position* changed.

Why one flow evaluation, not twelve. Backprop through the full ODE means 12 sequential 1.3 B forward passes with activations retained. Sampling k fresh each step means the LoRA still sees every timestep, at roughly rung13’s cost. $\hat{x}_0$ uses the sampler’s **own** `_pred_to_xstart`, so the rectified-flow parameterisation cannot drift from TRELIS.2’s.

4. Gradient health — the first thing to check

The gradient travels back through the texture decoder **and** a flow evaluation, both in bf16, to a LoRA sitting behind cross-attention’s softmax where gradients are already small. TRELIS v1 hit exactly this failure (fp16 flush-to-zero in `dec_mesh`, fixed with `LOSS_SCALE = 4096`, carried over here).

Logged every epoch, and probed at **t = 1.0, 0.75, 0.5, 0.25** before training starts — starvation is often timestep-dependent, so a single-timestep probe misleads:

metric	what it catches
per-target $\ dB\ $ for <code>to_q</code> / <code>to_kv</code> / <code>to_out</code>	which projection is starving
per-block $\ dB\ $, all 30	a dead early half, invisible in an aggregate
count of matrices at exactly zero	total death
min / median / max and the spread ratio	near-underflow <i>before</i> it becomes zero
$\ dB\ / \ B\ $	whether the step size is meaningful

Reported on **B**, not **A** — B is zero-init, so if B’s gradient is zero then A’s is zero by the chain rule and nothing can ever move. Checking A would mislead.

GATE-grad refuses to start training if any of the 90 matrices gets zero gradient, and names the dead ones. Failing in 60 s beats discovering it after four hours.

5. What decides success

Not PSNR from the training view — rung13 already scores well there and is still wrong off-axis. The metrics that matter:

1. **Turntable.** Does the texture hold up as the camera rotates away from the training view? rung13’s degrades badly by 180°.
2. **Temporal flicker**, frame to frame, against the per-frame baseline’s.
3. **Geometry unchanged** — free here, since TRELIS.2 takes the mesh as an input and cannot modify it.

6. Two implementation traps, both hit and fixed

Do not call `sampler._inference_model`. `FlowEulerGuidanceIntervalSampler` inherits two CFG mixins that override it with `(model, x_t, t, cond, guidance_strength, guidance_interval, **kwargs)` — both required. Omitting them raises `TypeError`. Call the flow directly and replicate `FlowEulerSampler`’s own $t \rightarrow 1000 \cdot t$ scaling.

`o_voxel`, `cumesh` and `nvdiffrast` need GLIBC ≥ 2.32 . The login node is 2.31; compute nodes are 2.35. Everything must run through `sbatch`.

7. Files

file

<code>rung14_crossattn_lora.py</code>	the LoRA, hooks, gates, gradient instrumentation
<code>jobs/rung14_gradprobe.sbatch</code>	the gradient probe
<code>run_baseline.py</code>	frozen per-frame TRELIS.2 — the comparison this is measured against

TRELIS.2’s own source is imported, never modified. `rung13` and everything before it is untouched.